NodeJS Notes

-Mo Fahim Raj

Introduction to Node.js

What is Node js

Nodejs is a runtime environment for javaScript. Node we can run JavaScript outside to the browser Like server, windows, mac , linux etc JavaScript run inside the browser.

Where we need JS outside browser

Web browser Run on client computer. So Javascript do not handle things on server. So we can not connect javascript with Database directly Can not handle File system etc

So we have to run javascript on browser and Node help for this

Why need Node?

Unified Language

High Performance

Good for Real-time Applications

Good for Data Streaming

How to use it full stack and Frontend technology

We make API in node js. Integrate APIs with Frontend technology.

Backend + Frontend = Full Stack.

History Of Node js

First Release :- May 27, 2009 Current Version: V22.16.0

Written in JS, C and c++

NodeJS Installation

Node.js Setup on MacOS

Step 1: Download Node.js

Go to the official Node.js website:

👉 https://nodejs.org/en

Click on **Download Node.js (LTS)** version.

(LTS means Long Term Support – best for most users.)

Step 2: Install Node.js

Open the downloaded file and complete the installation steps.

Once installed, Node.js and **npm (Node Package Manager)** will be available.

Check Installed Versions

Open **Terminal**.

Type the following commands to verify installation:

```
node -v
npm -v
```

This will show you the version numbers of Node.js and npm installed.

Node.js Setup on Windows

Step 1: Download Node.js

Go to:

👉 https://nodejs.org/en

Click on the **Download Node.js (LTS)** version.

Step 2: Install Node.js

Run the downloaded setup file and complete the installation.

This installs both **Node.js** and **npm**.

Step 3: Check Installed Versions

Open **Command Prompt (CMD)**.

Run these commands:

```
node -v
npm -v
```

You should see version numbers if everything was installed correctly.

Install VS Code Editor

To write and run Node.js code smoothly, install **Visual Studio Code** (VS Code) — a lightweight code editor.

For macOS:

Visit: https://code.visualstudio.com/

Click **Download for macOS** and follow the install instructions.

For Windows:

Same site: https://code.visualstudio.com/

Click **Download for Windows**.

Once installed, open VS Code and start coding in JavaScript and Node.js easily.

NOTES

Create Folder and File

Create Folder

On Desktop, create a folder manually (e.g., NodeApp)

or

Open Terminal and run:

```
mkdir NodeApp
cd NodeApp
```

Open Folder in VS Code

Using VS Code directly

Open **VS Code**

Go to **File > Open Folder**

Select your NodeApp folder

Using Terminal

```
code .
```

This opens the current folder in VS Code (if code command is configured).

Create a JavaScript File

Create a file named app.js

Node.js runs JavaScript files, so .js extension is required.

Write Some Basic Code

Basic Output

```
console.log(20 + 20); // Output: 40
```

Function Example

```
function fruit(item) {
```

```
console.log("Fruit name is " + item);
}
fruit("apple");
```

Variable Example

```
var a = 10;
var b = 20;

console.log(a + b); // Output: 30
```

Run Node.js Code

Steps:

Open terminal

Navigate to your project folder (if not already)

Run:

```
node app.js
```

File System Example (Just to Show)

```
var fs = require("fs"); // Import file system module

fs.writeFileSync("anil.txt", "My name is fahim");
```

This creates a file anil.txt and writes the content inside it.

REPL (Read Eval Print Loop)

Just type node in terminal to enter REPL mode

Then type JS code line by line

```
> 2 + 2
4
> .exit
```

What is Server-Side and Client-Side Scripting?

Server — where our code runs (backend)

Client — the part which the user accesses (frontend)

Server-side scripting language

The languages that execute on the server.

Examples: PHP, Java

Client-side scripting language

The languages that execute on the client (browser).

Example: JavaScript

How Does Node.js Use JavaScript?

Normally JavaScript works on the client side (browser).

Node.js allows us to run JavaScript on the server side also.

What Do We Exactly Do with Node.js?

Node js is mostly used for API development

API send send and receive data between any two language

After make API in node js , API can use in front technologies , Mobile technologies and any other platform

Why Do We Need APIs?

APIs allow different applications to communicate with each other.

Example:

Frontend app sends a request → API processes it on the server → API sends back the response (data) to frontend or mobile app.

That's why we build APIs — so that our backend and frontend (or mobile app) can work together.

How to Import Functions or Variables From Another File

Export from data.js:

```
module.exports = {
```

```
 userName: "fahim raj",
};
```

Import in basics.js:

```
const data = require("./data.js");


console.log(data.userName);
```

Why Do We Need a Server?

JavaScript by default runs in the browser.

But when we want to run JavaScript on the server, we need Node.js.

Now, if you write some code in Node.js — how can you access that code in the browser?

It's important to understand this:

If you want to see your Node.js output in the browser, you need to set up a server.

Write Code to Create a Server

Node.js provides a built-in package called http to create servers.

You don't need to install it separately.

Here's the basic code to create a server:

```
const http = require("http");


http.createServer().listen(4800);
```

How to Run the Code

Open your terminal and run the file using:

```
node basics.js
```

How to Check the Output in the Browser

Open your browser and type:

```
http://localhost:4800
```

You can also check the response in:

```
Inspect → Network tab
```

Understanding Request and Response Parameters

If you want your server to return some data to the browser, this is how it works:

> The **client** (browser) sends a request.
>
> The **server** responds to that request.

In Node.js, you can handle this using the http.createServer() method, which accepts a callback function with request and response parameters.

```
http
.createServer((req, resp) => {
resp.write("<h1>Hello, this is fahim raj</h1>");
resp.end("Hello"); // Ends the response
})
.listen(4800);
```

Now, when you visit http://localhost:4800, you will see the message in the browser.

What is an External Package?

An external package is a package we install from a third party (someone else wrote it).

We use it to add **extra functionality** to our project without writing everything ourselves.

Example:

nodemon — a package that automatically restarts the server when code changes.

Without this, we would have to stop and start the server manually after every change.

How to Install an External Package

Example: Install the colors package from https://www.npmjs.com/package/colors

Open the terminal and run:

```
npm install colors
```

or

```
npm i colors
```

Example of Using an External Package

```
const colors = require("colors");


console.log(colors.red("Hi, this is Anil Sidhu"));
console.log(colors.green("Hi, this is Anil Sidhu"));
console.log(colors.yellow("Hi, this is Anil Sidhu"));
```

To run this code, type in the terminal:

```
node colors.js
```

What is package.json?

package.json is a file that contains details about your project, such as:

- Project name
- Project version
- External packages used (dependencies)
- Project description
- Author
- License

What is package-lock.json?

package-lock.json is a file that contains detailed information about the exact versions of every installed package **and its dependencies** (internal packages).

It ensures that everyone working on the project installs **the same package versions**.

What is node_modules?

node_modules is a folder where all installed external packages are stored.

How to Create a package.json File

Run this command in the terminal:

```
npm init
```

You will be asked to fill in some details:

```
package name: node-2025
version: (1.0.0)
description: node js tutorial in hindi 2025
entry point: (index.js)
git repository:
keywords:
author: Anil Sidhu
license: (ISC)
```

Important Notes

External packages are stored inside the node_modules folder.

You should not push node_modules to GitHub — because it is a very large folder.

Other developers can easily install the required packages by running:

```
npm install
```

This will read package.json and package-lock.json and install everything needed.

Why do we need the nodemon package?

When you're working with a Node.js server, every time you make a change in your server file, you have to:

Manually stop the server.

Restart it to see the changes.

This process becomes annoying during development

Solution: nodemon

nodemon automatically **watches your files**.

It **restarts the server** automatically whenever you make changes.

Saves **time** and improves **developer experience**.

How to Install Nodemon

Open your terminal and run:

```
npm install nodemon
```

How to Use Nodemon

```
nodemon app.js
```

Now the server will restart automatically every time you make changes to app.js

Can we create more than one server?

Yes, we can run multiple servers in Node.js — just make sure they run on **different ports**.

What is a Server Response?

When a **client** (like a browser) sends a request to a server (e.g., by visiting google.com), the **server processes the request** and sends back a **response**.

This response can be **HTML**, **JSON**, **plain text**, files, or anything else.

Example:

When you visit **google.com**, the server sends back HTML that builds the **Google homepage** on your screen.

Basic Example of Server Response

```
const http = require("http");

http
.createServer((req, resp) => {
  resp.setHeader("Content-Type", "text/html"); // Tell browser the response is HTML
  resp.write("<h1>Hello Fahim Raj</h1>");
  resp.end(); // End the response
})
.listen("4800");
```

How to run:

Save as app.js

Run: nodemon app.js or node app.js

Open your browser and go to: http://localhost:4800

Passing Variables and Functions in Response

You can also include **variables**, **functions**, or **dynamic values** (like current time):

```
const http = require("http");

const age = 29;

http
 .createServer((req, resp) => {
  resp.setHeader("Content-Type", "text/html");

  resp.write(`
   <html>
    <head>
     <title>Code Step by Step</title>
    </head>
    <body>
     <h1>Hello Fahim Raj</h1>
     <h2>Age: ${age}</h2>
     <h2>Current Date & Time: ${Date()}</h2>
    </body>
   </html>
  `);

  resp.end();
 })
 .listen(4800);
```

resp.setHeader("Content-Type", "text/html"): Tells the browser that we're sending back HTML.

Template literals (` `): Used to inject JavaScript variables like age and Date() into the HTML.

Create Server in simpleAPI.js

We'll use Node.js http module to create a basic server.

Define Static API Data

```
const userData = [
{
```

```
name: "Anil",
age: 30,
email: "anil@test.com",
},
{
name: "sam",
age: 20,
email: "sam@test.com",
},
{
name: "peter",
age: 30,
email: "peter@test.com",
},
];
```

This is sample static data to simulate API response.

Use This Data in Server Response

```
const http = require("http");

http
.createServer((req, resp) => {
resp.setHeader("Content-Type", "application/json"); // Set the content type to JSON
resp.write(JSON.stringify(userData)); // Convert the userData array to a JSON string
resp.end(); // End the response
})
.listen(6000);
```

How to Test the API

Run the file using:

```
nodemon simpleAPI.js
```

Open browser and go to:

```
http://localhost:6000
```

Test API using Thunder Client

You can also use Thunder Client (VS Code extension) or Postman:

Method: GET
URL: http://localhost:6000

You will get the JSON response.

What is a Client Request?

A **client request** is the action performed by the client (browser or any API testing tool) when it tries to access a URL or send data to the server.

Whenever you open a website or call an API, you're sending a **request** to the **server**, and in return, the server sends a **response**.

Code to Access Client Request

```
const http = require("http");

http
 .createServer((req, resp) => {
  console.log(req.url); // Log the requested URL
  console.log(req.headers); // Log the request headers
  console.log(req.headers.host); // Log the host
  console.log(req.method); // Log the HTTP method (GET, POST, etc.)
 })
 .listen(5000);
```

Now open localhost:5000/, and check the terminal — you'll see the request data being logged.

Make Pages with Client Request

```
const http = require("http");

http
 .createServer((req, resp) => {
  if (req.url == "/") {
   resp.write("<h1>Home page</h1>");
```

```
  } else if (req.url == "/login") {
    resp.write("<h1>Login page</h1>");
  } else {
    resp.write("<h1>Other page</h1>");
  }
  resp.end(); // End the response
})
.listen(5000);
```

What is Command Line Input?

Sometimes, we want to pass values **directly while running a file** from the terminal — like a username or a dynamic port number.

Create File: command-line-input.js

```
nodemon command-line-input.js fahim raj
```

This runs the file and passes "fahim" and "raj" as input arguments.

How to Get Command Line Input

```
const arg = process.argv;


console.log("-------", arg);
Output:------- [
 'C:\\Program Files\\nodejs\\node.exe',
 'C:\\Users\\sheet\\OneDrive\\Desktop\\turorial\\node 2025\\command-line-input.js',
 'fahim',
 'raj'
]
```

🧾 process.argv is an array:

Index 0 → Node path

Index 1 → File path

Index 2 onwards → Actual input values

Access Individual Input

```
console.log("-------", arg[3]);
```

Output:

```
------- raj
```

Note: Space-separated values are stored as separate elements in the array.

Example Use Case: Dynamic Port

```
const http = require("http");
const arg = process.argv;
const port = arg[2];

http
 .createServer((req, resp) => {
   resp.write("test input from cmd");
   resp.end();
 })
 .listen(port);
```

Send HTML File in Response using FS Module

Why load an HTML file with fs?

Instead of writing HTML as a string in resp.write(), it's more scalable and readable to keep HTML in a separate .html file.

This approach is cleaner, especially for large or structured HTML.

Server File (web.js)

```
const http = require("http");

http.createServer((req, resp) => {}).listen(3200);
```

Creates a valid HTTP server.

Listens on port 3200.

HTML File (web.html)

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Web page</title>
 </head>
 <body>
  <h1>Web Page Heading</h1>
  <input type="text" />
  <button>Click</button>
 </body>
</html>
```

Well-formed HTML structure.

Proper title, meta tags, and UI elements.

Read and Load HTML File Using fs Module

```
const http = require("http");
const fs = require("fs");

http
 .createServer((req, resp) => {
  fs.readFile("html/web.html", "utf-8", (err, data) => {
    if (err) {
      resp.writeHead(500, {
        "Content-Type": "text/plain",
      });
      resp.write("internal server error");
      resp.end();
      return;
    }

    resp.writeHead(200, {
      "Content-Type": "text/html",
    });
    resp.write(data);
    resp.end();
```

```
  });
}).listen(3200);
```

Correctly reads the file html/web.html (assuming the folder structure is correct).

Sets appropriate status codes (500 for error, 200 for success).

Handles errors gracefully.

Sends the HTML content as response.

How to Submit Form with Node.js

Form inside JS file

You correctly created and served a basic HTML form directly via resp.write():

```
const http = require("http");

http
.createServer((req, resp) => {
  resp.writeHead(200, {
    "Content-Type": "text/html",
  });
  resp.write(`
    <form action="/submit" method="POST">
      <input type="text"  placeholder="Enter username" name="name" />
      <input type="text"  placeholder="Enter email" name="email" />
      <button>Submit</button>
    </form>
  `);
  resp.end();
})
.listen(3200);
```

This works fine, though you're not handling the actual POST data yet — just confirming routing works.

Conditional Handling of /submit

```
if (req.url === "/") {
 // form page
} else if (req.url === "/submit") {
 // confirmation message
}
```

This conditional routing is correct for handling basic URL-based navigation.

Using a separate form.html file

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Form page</title>
</head>
<body>
 <form action="/submit" method="POST">
  <input type="text" placeholder="Enter username" name="name" />
  <input type="text" placeholder="Enter email" name="email" />
  <button>Submit</button>
 </form>
</body>
</html
```

Perfectly structured form with POST method and inputs.

Load HTML File via fs

```
const http = require('http');
const fs = require('fs');

http.createServer((req, resp) => {
 fs.readFile('html/form.html', 'utf-8', (error, data) => {
  if (error) {
   resp.writeHead(500, { "content-type": 'text/plain' });
   resp.end('internal server error');
   return;
```

```
  }

  resp.writeHead(200, { "content-type": 'text/html' });

  if (req.url == '/') {
    resp.write(data);
  } else if (req.url == '/submit') {
    resp.write('<h1>Data submitted</h1>');
  }

  resp.end();
});
}).listen(3200);
```

Handle Form Request in Nodejs

When a form is submitted using the POST method, data is sent in chunks. To handle this in Node.js, we use two important events: on('data') and on('end').

Get data from request

We collect incoming data chunks into a buffer:

```
let dataBody = [];
req.on("data", (chunk) => {
 dataBody.push(chunk);
});
```

Handle data with Buffer

Once all data is received, we combine and convert it to a string:

```
req.on("end", () => {
 let rawData = Buffer.concat(dataBody).toString();
 console.log(rawData);
});
```

```
resp.write(`<h1> Data Submitted </h1>`);
```

Make request data readable

To parse this string into key-value pairs, use Node's built-in querystring module:

```
const querystring = require("querystring");

req.on("end", () => {
 let rawData = Buffer.concat(dataBody).toString();
 let readableData = querystring.parse(rawData);
 console.log(readableData);
});
```

Notes

on('data') collects the chunks

on('end') finalizes the data

querystring.parse() makes the data easier to use

Create File with Requested Form Data

When users submit form data, we can save it in a file using Node.js. There are two ways to create files: **Synchronous** and **Asynchronous**.

Synchronous File Creation

```
let dataString =
 "My name is " +
 readableData.name +
 " and my email is " +
 readableData.email;

fs.writeFileSync("text/" + readableData.name + ".txt", dataString);
console.log("File Created");
```

dataString: combines name and email from the form.

writeFileSync: writes data to file in a blocking way (waits till complete).

"text/" + readableData.name + ".txt": file name is based on user input.

Asynchronous File Creation

```
fs.writeFile(
 "text/" + readableData.name + ".txt",
 dataString,
 "utf-8",
 (err) => {
  if (err) {
   resp.end("Internal Server Error");
   return false;
  } else {
   console.log("File Created");
  }
 }
);
```

writeFile: non-blocking (code runs while file is being created).

"utf-8": encoding type.

Callback handles success or error.

How Node Works Behind the seen Node.js

Let's understand how Node.js works behind the scenes — especially how the **Call Stack**, **Node APIs**, and **Event Loop** come together.

Main Function Execution

When Node.js runs code, it first registers the **main()** function in the **call stack**. Then other functions are called in order.

```
const x = 1;
const y = x + 2;
console.log("sum is " + y);
```

main() is added to the call stack automatically.

After console.log(), it exits the stack.

Only main() remains and exits last.

Example with a Function

```
const listLocations = (locations) => {
```

```
 locations.forEach((location) => {
   console.log(location);
 });
};

const myLocations = ["philly", "nyc"];

listLocations(myLocations);
```

Call Stack Flow:

listLocations enters call stack
forEach runs each item
console.log("philly")
Stack clears, then next item:
console.log("nyc")
Stack becomes empty

Node APIs & setTimeout

Node APIs handle asynchronous code like setTimeout.

```
console.log("Starting up");

setTimeout(() => {
 console.log("Two Seconds");
}, 2000);

setTimeout(() => {
 console.log("Zero Seconds");
}, 0);

console.log("Finishing up");
```

Call Stack Output Order:

Starting up
Finishing up
(then Node APIs hold timers)

Event Loop then executes callbacks when the call stack is empty:

3. Zero Seconds

4. Two Seconds

Event Loop

Keeps checking if the **call stack** is empty.

If empty, moves code from **callback queue** to **call stack**.

How to make custom modules in node js

Node.js is great for creating modular server-side applications. In this post, we'll create a simple project using **modules**, **routing**, and **form handling**.

Project Structure

Create 3 files:

root.js (main server file)

userForm.js (HTML form)

userDataSubmit.js (handle form data)

root.js (Main Server File)

```
const http = require("http");
const userForm = require("./userForm");
const userDataSubmit = require("./userDataSubmit");

http
 .createServer((req, resp) => {
  resp.writeHead(200, { "Content-Type": "text/html" });

  if (req.url === "/") {
    userForm(req, resp);
  } else if (req.url === "/submit") {
    userDataSubmit(req, resp);
  }

  resp.end();
```

```
})
.listen(3200);
```

This file:

Creates the HTTP server

Routes / to form and /submit to form handler

Sends HTML response based on the route

userForm.js (HTML Form)

```
function userForm(req, resp) {
 resp.write(`
  <form action="/submit" method="POST">
    <input type="text" placeholder="Enter username" name="name" />
    <input type="text" placeholder="Enter email" name="email" />
    <button>Submit</button>
  </form>
 `);
}

module.exports = userForm;
```

This file:

Displays a form with two input fields

Sends data to /submit via POST method

userDataSubmit.js (Handle Form Data)

```
const querystring = require("querystring");

function userDataSubmit(req, resp) {
 let dataBody = [];

 req.on("data", (chunk) => {
   dataBody.push(chunk);
 });

 req.on("end", () => {
```

```
  let rawData = Buffer.concat(dataBody).toString();
  let readableData = querystring.parse(rawData);

  let dataString =
    "My name is " +
    readableData.name +
    " and my email is " +
    readableData.email;

  console.log(dataString);
});

 resp.write(`<h1>You can get data from user form here</h1>`);
}

module.exports = userDataSubmit;
```

This file:

- Listens to data from the form
- Collects data chunks using Buffer
- Converts it to a readable object using querystring
- Logs name and email in the console

Output Demo

```
node root.js
```

Open browser at:

http://localhost:3200/

CRUD with File System Create, Read, Update, Delete Files

Learn how to perform **Create, Read, Update, and Delete** operations using Node.js's built-in fs module. We'll also handle terminal inputs to do these operations dynamically.

Basic File Operations

```
const fs = require("fs");
```

Create New Files

```
fs.writeFileSync("files/apple.txt", "This is fruit");
fs.writeFileSync("files/banana.txt", "This is fruit");
```

writeFileSync() creates files. If file already exists, it will overwrite.

Delete a File

```
fs.unlinkSync("files/banana.txt");
```

unlinkSync() removes a file.

Read a File

```
const data = fs.readFileSync("files/apple.txt", "utf-8");
console.log(data);
```

readFileSync() reads file content. "utf-8" ensures readable string output.

Update a File

```
fs.appendFileSync("files/apple.txt", " and this is good for health");
```

appendFileSync() adds new content to existing file.

CRUD Operations with Terminal Input

Now let's make this dynamic — use terminal commands like Write, Read, update, or delete.

```
const fs = require("fs");
const operation = process.argv[2];
```

process.argv[] gives command-line input.

Create File from Terminal

```
node index.js Write fileName "your content"
if (operation === "Write") {
```

```
const name = process.argv[3];
const content = process.argv[4];
fs.writeFileSync(`files/${name}.txt`, content);
console.log("File Created");
}
```

Read File from Terminal

```
node index.js Read fileName
else if (operation === "Read") {
const name = process.argv[3];
const data = fs.readFileSync(`files/${name}.txt`, "utf-8");
console.log(data);
}
```

Update File from Terminal

```
node index.js update fileName "new content"
else if (operation === "update") {
const name = process.argv[3];
const content = process.argv[4];
fs.appendFileSync(`files/${name}.txt`, content);
}
```

Delete File from Terminal

```
node index.js delete fileName
else if (operation === "delete") {
const name = process.argv[3];
fs.unlinkSync(`files/${name}.txt`);
}
```

Fallback Case

```
else {
console.log("Operation not found");
}
```

Example Test

```
node index.js Write hello "This is demo"
node index.js Read hello
node index.js update hello " updated!"
node index.js delete hello
```

Synchronous vs Asynchronous Programming in node and Javascript

In most programming languages, operations run **synchronously** — one task at a time. But in JavaScript and Node.js, we often use **asynchronous programming** for better speed and performance.

Synchronous Programming

Executes **one task at a time**.

The next line waits until the current one finishes.

Slower in terms of overall performance.

```
console.log("Apple1");
console.log("Apple2");
console.log("Apple3");
```

Each task waits for the previous one to complete.

Asynchronous Programming

Doesn't wait for a task to finish before moving on.

Great for **I/O operations** (file, API, database).

Faster in real-world applications.

```
console.log("Apple1");

setTimeout(() => {
 console.log("Apple2");
}, 2000);

console.log("Apple3");
```

setTimeout() runs separately and doesn't block the next line.

When Not to Use Asynchronous Code?

Sometimes, we want code to **wait** until one step is complete.

```
let a = 20;
let b = 0;

setTimeout(() => {
 b = 100;
}, 2000);

console.log(a + b); // Output: 20 (Not 120!)
```

Here, we **don't need** async because we want b before using it.

Real-Life Async Example

Reading a file without blocking the main thread:

```
onst fs = require("fs");

fs.readFile("text/peter.txt", "utf-8", (err, data) => {
 if (err) return false;
 console.log(data);
});

console.log("End Script");
```

Read FIle in Sync

```
const data = fs.readFileSync("text/peter.txt", "utf-8");

console.log(data);
```

Make Simple and Basic Website

Create a basic website with **HTML and CSS** using **Node.js**. We'll load different HTML pages and a CSS file using the fs (File System) module and http core module.

Create a server file

website.js

```
const http = require("http");

http
 .createServer((req, resp) => {
  resp.write("page check");
  resp.end();
 })
 .listen(3200);
```

This creates a simple HTTP server.

Load HTML file using fs

```
const http = require("http");
const fs = require("fs");

http
 .createServer((req, resp) => {
  fs.readFile("html/home.html", "utf-8", (error, data) => {
   if (error) {
    resp.writeHead(500, { "content-type": "text/plain" });
    resp.end("Internal Server Error");
    return false;
   }

   resp.write(data);
   resp.end();
  });
 })
 .listen(3200);
```

This loads the home.html file and displays its content on the browser using **asynchronous** fs.readFile().

Sync vs Async in fs (Used in Header)

In the code below, we use:

```
let collectHeaderData = fs.readFileSync("html/header.html", 'utf-8');
```

Synchronous (Sync)

fs.readFileSync() blocks further execution until the file is fully read.

Use it for small files like header.html where immediate availability is needed.

Later in the same code:

```
fs.readFile("html"+file+".html", 'utf-8', (error, data) => {
 ...
});
```

Asynchronous (Async)

fs.readFile() does **not block** the code. It continues and runs the callback when data is ready.

Better for performance, especially for larger or external files.

Handle HTML and CSS File Together

```
const http = require('http');
const fs = require('fs');

http.createServer((req, resp) => {

  //  Sync: Header file loaded before anything else
  let collectHeaderData = fs.readFileSync("html/header.html", 'utf-8');

  let file = "/home";
  if (req.url !== '/') {
     file = req.url;
  }

  // Async: Load HTML file with callback
  if (req.url !== '/style.css') {
     fs.readFile("html" + file + ".html", 'utf-8', (error, data) => {
        if (error) {
           resp.writeHead(500, { "content-type": "text/plain" });
           resp.end("internal server error");
```

```
            return false;
        }

        resp.write(collectHeaderData + "" + data);
        resp.end();
    });
  }
  // Serve CSS with correct content-type
  else if (req.url === '/style.css') {
    fs.readFile("html/style.css", 'utf-8', (error, data) => {
        if (error) {
            resp.writeHead(500, { "content-type": "text/plain" });
            resp.end("css not found");
            return false;
        }

        resp.writeHead(200, { "content-type": "text/css" });
        resp.end(data);
    });
  }

}).listen(3200);
```

Path Module and Path global constant

What is the Path module?

In Node.js, the **path module** is a built-in core module that provides utilities for working with file and directory paths.

What is Global constant?

In Node.js, a **global constant** is a value that is always available in your code without requiring an import or declaration.

Example of both:

```
const path = require("path");
const file = "text/peter.txt";

console.log(path.extname(file)); // Output: .txt
console.log(path.dirname(file)); // Output: text
console.log(path.basename(file)); // Output: peter.txt
console.log(path.resolve("text", "peter.text")); // Output: absolute path to text/peter.text
console.log(path.isAbsolute(file)); // Output: false (if the path is not absolute)
```

This are the global constants in Node.js

```
console.log(__dirname); // Output: current directory path
console.log(__filename); // Output: current file path
```

Why do we need global constants?

This is very helpful when you're working with file systems (like reading HTML or CSS files), especially when your project is running on different machines or environments. It ensures your file paths are always accurate.